

Computer Hardware

Performance and Pipelining

Ksawery Moźdzynski

Agenda For Today

1. Measuring Performance

Execution time, CPI, the iron law and benchmarks

2. From MARIE to RISC-V

A real instruction set, the Ripes simulator and a single-cycle processor

3. Pipelining

Many instructions in the processor at the same time

4. Branches

When the pipeline does not know what comes next

Today you will need: Ripes

In the browser



<https://ripes.dk>
nothing to install

Desktop app



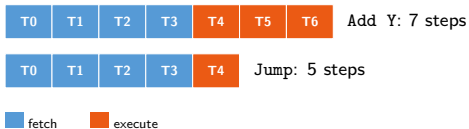
github.com/mortbopet/Ripes/releases
Windows, macOS, Linux

Linux: `chmod a+x Ripes-*.AppImage`, then run it. If you see an error about FUSE on Ubuntu: `sudo apt install libfuse2t64`.

Please open Ripes now: we will use it in about an hour. It is a free and open-source RISC-V simulator.

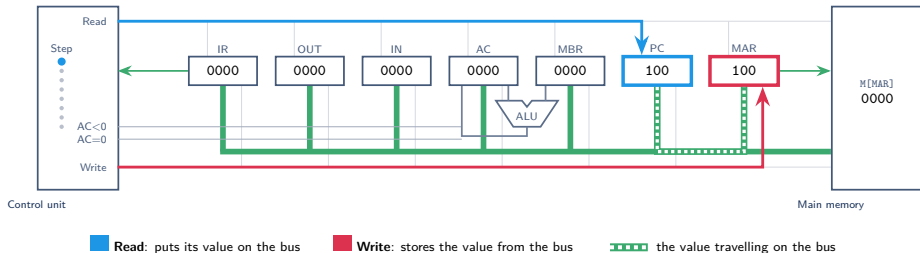
Day 2 recap: how long does one instruction take?

Instruction	Fetch	Execute	Total steps
Load X, Add Y, Store Z	4	3	7
Output, Jump	4	1	5



Every instruction = **the same fetch** + **its own execute steps**.

Day 2 recap: the control unit



In every step

the control unit sets the Read and Write signals. Here:
 $MAR \leftarrow PC$.

Two ways to build it

hardwired (logic gates) or **microprogrammed** (a small program inside the CPU)

Day 2 recap: RISC vs CISC

RISC



- + few, simple instructions of fixed length: easy to decode
- more instructions per program

CISC



- + rich instructions of variable length: fewer instructions per program
- harder to decode, instructions take different times

Is this processor fast?

Two single-core ARM processors for microcontrollers:

Arm Cortex-M3



Introduced: 2004
Clock: 120 MHz

Arm Cortex-M7

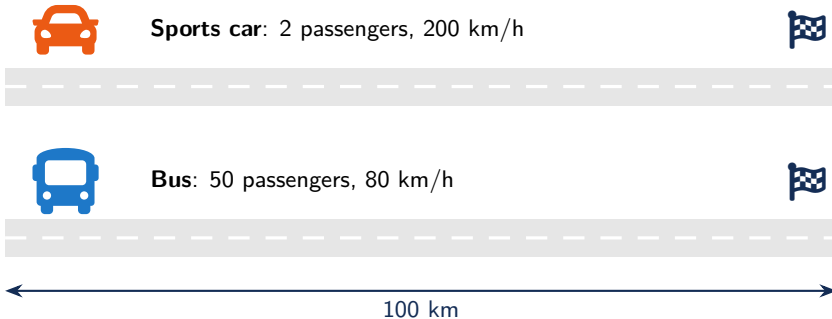


Introduced: 2014
Clock: 100 MHz

Which one is faster? What does “faster” even mean?

Before we can measure a processor, we must agree what “faster” means. Let's start with something simpler than a computer.

Which one is faster?



Which one is faster?

Two meanings of “faster”

	Time of one trip	Passengers per hour
 Sports car	0.5 h	4
 Bus	1.25 h	40

(we ignore the trips back)

Response time (latency)

How long does **one** job take?

Throughput

How **many** jobs are finished per unit of time?

The car wins on latency, the bus wins on throughput.

A processor is a vehicle

 Vehicle	 Processor
passengers to carry	instructions of a program
passengers per trip	instructions per clock cycle
trips per hour	clock rate
time to carry everyone	execution time of the program

Carry 10 000 passengers

 car, 4 per hour: **2500 h**

 bus, 40 per hour: **250 h**

Every bus trip is slower, but the bus finishes the job **10 times** sooner.

A program means carrying **millions** of passengers. What matters is how long it takes to carry **them all**.

How long does program B run?

Let's assume we have a **processor A** with a clock rate of **2 GHz**. We want to run **program B** on it, whose **instruction count** (IC, the number of instructions it executes) is 2×10^9 . To run program B, processor A needs 6×10^9 **clock cycles**.

Processor A



clock rate: 2 GHz

Program B



IC: 2×10^9 instructions
 6×10^9 clock cycles

How long will processor A take to run program B?

Where does the time go?

Processor A: 2 GHz · Program B: IC = 2×10^9 instructions, 6×10^9 clock cycles

$$\text{CPU time} = \frac{\text{clock cycles}}{\text{clock rate}} \quad \frac{6 \times 10^9}{2 \times 10^9 \text{ /s}} = \mathbf{3 \text{ s}}$$

Where does the time go?

Processor A: 2 GHz · Program B: IC = 2×10^9 instructions, 6×10^9 clock cycles

$$\text{CPU time} = \frac{\text{clock cycles}}{\text{clock rate}}$$

$$\frac{6 \times 10^9}{2 \times 10^9 \text{ /s}} = 3 \text{ s}$$

$$\text{CPI}_{\text{avg}} = \frac{\text{clock cycles}}{\text{IC}}$$

CPI_{avg} = average clock cycles per instruction

$$\text{CPI}_{\text{avg}} = \frac{6 \times 10^9}{2 \times 10^9} = 3$$

Where does the time go?

Processor A: 2 GHz · Program B: IC = 2×10^9 instructions, 6×10^9 clock cycles

$$\text{CPU time} = \frac{\text{clock cycles}}{\text{clock rate}}$$

$$\frac{6 \times 10^9}{2 \times 10^9 / \text{s}} = 3 \text{ s}$$

$$\text{CPI}_{\text{avg}} = \frac{\text{clock cycles}}{\text{IC}}$$

CPI_{avg} = average clock cycles per instruction

$$\text{CPI}_{\text{avg}} = \frac{6 \times 10^9}{2 \times 10^9} = 3$$

$$\text{so CPU time} = \frac{\text{IC} \times \text{CPI}_{\text{avg}}}{\text{clock rate}}$$

$$\frac{2 \times 10^9 \times 3}{2 \times 10^9 / \text{s}} = 3 \text{ s} \quad \checkmark$$

Why an average? In MARIE Add Y takes 7 cycles and Jump takes 5: different instructions need different numbers of cycles.

The iron law of performance

$$\text{CPU time} = \frac{\text{IC} \times \text{CPI}_{\text{avg}}}{\text{clock rate}}$$

IC

instruction count:
how many instructions run

👤 how many passengers

CPI_{avg}

average clock cycles
per instruction

🚗 1 / passengers per trip

Clock rate

clock cycles per second
1 GHz = 10⁹ cycles/s

🕒 trips per hour

From now on we write simply **CPI**, as textbooks do.

How can we make a program faster?

$$\text{CPU time} = \frac{\text{IC} \times \text{CPI}}{\text{clock rate}}$$

IC ↓
fewer instructions

CPI ↓
fewer cycles per instruction

Clock rate ↑
more cycles per second

Improve **one** of the three
without making the other two worse.

What changes each of the three?



- the algorithm
- the compiler
- the instruction set (RISC or CISC)



- the organisation of the processor: datapath, control unit
- e.g. **pipelining** (today!)



- technology: faster transistors
- organisation: less work in each step

Exercise 1: CPI of a MARIE program

Task

A MARIE program executes 10^9 instructions on a 1 GHz clock.

Instruction	Share	Cycles
Load	30%	7
Add	30%	7
Store	20%	7
Jump	10%	5
Output	10%	5

1. What is the **average CPI**?

2. What is the **execution time**?

Hint: $\text{average CPI} = \sum_i \text{share}_i \times \text{cycles}_i$

1 GHz = 10^9 clock cycles per second

Solution: CPI of a MARIE program

1. Average CPI

$$\text{CPI} = (0.3 + 0.3 + 0.2) \times 7 + (0.1 + 0.1) \times 5 = 5.6 + 1.0 = \mathbf{6.6}$$

2. Execution time

$$\text{CPU time} = \frac{\text{IC} \times \text{CPI}}{\text{clock rate}} = \frac{10^9 \times 6.6}{10^9 / \text{s}} = \mathbf{6.6 \text{ s}}$$

Exercise 2: the megahertz myth

Task

Remember the two ARM processors from the beginning? The same program (10^9 instructions) and the same instruction set on two processors. The shop says: “B has a 3 GHz clock instead of 2 GHz, so it is 1.5 times faster.” Is it true? Which one is faster, and how many times?

Processor A



Clock: 2 GHz
CPI: 2.0

Processor B



Clock: 3 GHz
CPI: 3.5

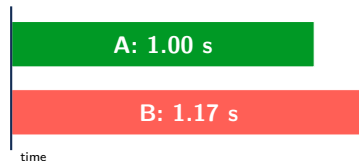
Solution: the megahertz myth

$$\text{A: } \frac{10^9 \times 2.0}{2 \times 10^9} = 1.00 \text{ s}$$

$$\text{B: } \frac{10^9 \times 3.5}{3 \times 10^9} = 1.17 \text{ s}$$

$1.17/1.00 \approx 1.17$: **A is 1.17 times faster.**

The shop only divided the clock rates ($3/2 = 1.5$). It ignored CPI.



B has a faster clock but a worse CPI: improving one of the three is not enough if another one gets worse.

A popular number: MIPS

$$\text{MIPS} = \frac{\text{Instruction count}}{\text{Time} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}$$

millions of instructions per second

Why is it tempting?

One number, and bigger means better.

A and B from Exercise 2

A: $2000 / 2.0 = 1000 \text{ MIPS}$

B: $3000 / 3.5 = 857 \text{ MIPS}$

MIPS agrees with time here.

Does MIPS **always** agree with execution time? Let's check.

Exercise 3: can MIPS lie?

Task

Two processors, both 2 GHz, run the same program compiled for two **different instruction sets**. For each, compute the **execution time** and the **MIPS**. Which one is faster?

Processor	Instruction count	CPI	Clock
C (CISC)	0.6×10^9	3.0	2 GHz
D (RISC)	2.0×10^9	1.0	2 GHz

Solution: MIPS can lie

	Time	MIPS
C (CISC)	$\frac{0.6 \times 10^9 \times 3}{2 \times 10^9} = \mathbf{0.90\ s}$	667
D (RISC)	$\frac{2.0 \times 10^9 \times 1}{2 \times 10^9} = \mathbf{1.00\ s}$	2000

D has 3× more MIPS...

...but C finishes the program first.

Why?

MIPS ignores how much work one instruction does.
Remember RISC vs CISC from Day 2.

Never compare MIPS across different instruction sets. **Compare time.**

How to compare fairly: benchmarks

Benchmark

A real test program that we run on both processors. It measures **real work**, not the number of instructions, so it cannot lie like MIPS.

Examples

SPEC CPU for computers, **CoreMark** for microcontrollers.

CoreMark score

how many times per second
the test program runs



÷ clock rate in MHz

CoreMark/MHz

how many times the test program runs
in **one million clock cycles**

CoreMark/MHz lets us compare cores **independently of their clock rate**. Higher is better.

Back to the two ARM processors

	Clock rate	CoreMark/MHz	CoreMark score
Arm Cortex-M3 (2004)	120 MHz	3.45	$120 \times 3.45 \approx 414$
Arm Cortex-M7 (2014)	100 MHz	5.29	$100 \times 5.29 \approx \mathbf{529}$

Source: Arm Cortex-M Processor Comparison Table (2022), documentation-service.arm.com/static/61bb37962183326f2176f8cc

$529/414 \approx \mathbf{1.28}$: the newer core is faster, with a **slower** clock.

Both cores use the same instruction set, so IC is about the same: the Cortex-M7 wins because it needs **fewer cycles per instruction**.

Exercise 4: write your own benchmark

Task

Write a short Python program that does a **fixed amount** of work (for example arithmetic, strings or lists in a loop repeated N times) and measures **how long** it takes.

1. Choose N so that the program runs a **few seconds** on your computer. Write N into the program.
2. Run it 2–3 times. Write down the time, your **processor model**, its **clock rate** and your **Python version** (`python --version`).
3. Swap programs with your neighbour. Run **both** programs on **both** computers.
 - How many times faster was one computer than the other in the first benchmark? And in the second?
 - Did the same computer win in both benchmarks?
 - Was the advantage similar in both, or clearly different?

Tips: `time.perf_counter()` gives the current time in seconds: read it before and after the work. Plug in your laptop and close other programs before you measure.

Solution: an example benchmark



```
1 import sys, time
2
3 N = 30_000_000          # fixed amount of work
4
5 start = time.perf_counter()
6 total = 0
7 for i in range(N):
8     total += i * i % 7
9 end = time.perf_counter()
10
11 print("Python", sys.version.split()[0])
12 print(f"Time: {end - start:.2f} s")
```

Fixed work, measured time

The same N on every computer, so we compare the time of the same program.

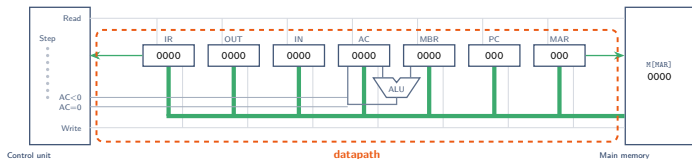
Real benchmarks do the same

SPEC CPU and CoreMark also run a fixed amount of work and measure the time.

Different benchmarks, different results

The winner, or the size of the advantage, can change: a benchmark measures only the work it contains. That is why real suites like SPEC use many programs.

What is a datapath?



MARIE from Day 2: the **datapath** between the control unit and main memory

Datapath

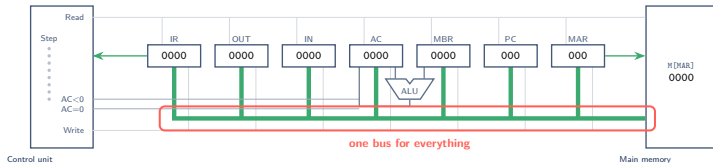
The part of the processor where data **flows** and is **processed**: registers, ALU, connections to memory, buses and wires.

Control unit

Tells the datapath **what to do** in every clock cycle: which register to read, what the ALU computes, whether to write.

Processor = **datapath** + **control unit**.

Why leave MARIE?



One bus

Only one value can travel at a time.

One accumulator

Almost every instruction needs AC, so instructions always wait for each other.

What we need

Separate paths for instructions and data, and many registers.

We switch to **RISC-V**: the RISC idea from Day 2 made real.

Meet RISC-V

32 registers

x0 = 0
x1
x2
x3
x4
⋮
x30
x31

32 registers

x0 ... x31, 32 bits each. x0 is always 0.

Load/store architecture

Memory is used only by lw (load word) and sw (store word). Arithmetic works on registers.

Fixed length

Every instruction is exactly 32 bits long.

An **open** instruction set: used in teaching, in microcontrollers, and more and more in industry.

The same program twice: $Z = X + Y$

MARIE



```
Load X
Add Y
Store Z
Halt
X, Dec 1
Y, Dec 2
Z, Dec 0
```

RISC-V



```
.data
X: .word 1
Y: .word 2
Z: .word 0
.text
lw  x5, X           # x5 = X
lw  x6, Y           # x6 = Y
add x7, x5, x6      # x7 = X + Y
sw  x7, Z, x28      # Z = x7
```

Three more instructions

Instruction	Meaning
<code>addi x7, x7, -2</code>	$x7 \leftarrow x7 - 2$ (a constant inside the instruction)
<code>beq x5, x6, label</code>	if $x5 = x6$, jump to label
<code>bnez x5, label</code>	if $x5 \neq 0$, jump to label

Already known: `lw`, `sw`, `add`

`beq` and `bnez` are **branches**: MARIE's `Skipcond` and `Jump` in one instruction.

Now a simulator is the CPU: Ripes

We will run our RISC-V programs in a simulator, in the browser:



`https://ripes.dk`

Exercise 5: your first RISC-V program

Ripes task

Write $Z = (X + Y) - 2$ in RISC-V and run it in Ripes.

- Put X: .word 5, Y: .word 3 and Z: .word 0 in the .data part.
- Use only the instructions we have seen: lw, sw, add, addi.
- Before running, choose the **single-cycle processor** (🔧).

Day 2

You solved this in “RISC style” with load, add, sub and store. Now it is a real instruction set.

Solution: your first RISC-V program



```
.data
X: .word 5
Y: .word 3
Z: .word 0
.text
lw    x5, X           # x5 = X
lw    x6, Y           # x6 = Y
add   x7, x5, x6       # x7 = X + Y
addi  x7, x7, -2       # x7 = x7 - 2
sw    x7, Z, x28       # Z = x7
```

Exercise 6: your first branch

Ripes task

Write a program that sets $Z = 1$ if $X = Y$, and $Z = 0$ otherwise.

- Use `beq` from the table: if two registers are equal, jump to a label.
- Run it twice: with $X = 5$, $Y = 5$ and with $X = 5$, $Y = 3$. Check Z in the **Memory** tab.

Day 2

In MARIE you did this with `Skipcond`: skip the next instruction if a condition is true.

Hint: a label marks a line: `done: sw x7, Z, x28.`
`beq x5, x6, done` jumps there.

Solution: your first branch



```
.data
X: .word 5
Y: .word 5
Z: .word 0
.text
    lw    x5, X
    lw    x6, Y
    addi  x7, x0, 1      # guess: equal, Z = 1
    beq   x5, x6, done   # if X = Y, skip next line
    addi  x7, x0, 0      # not equal, Z = 0
done: sw   x7, Z, x28
```

X = 5, Y = 5

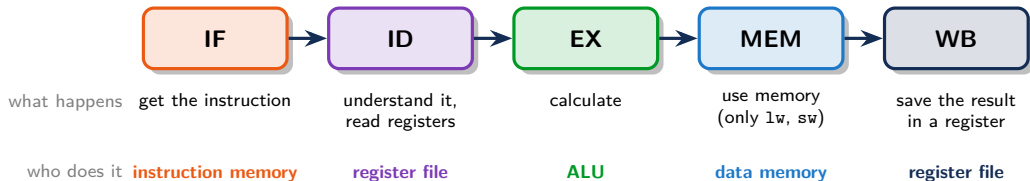
beq jumps to done: Z = 1

X = 5, Y = 3

no jump, the next line runs: Z = 0

beq decides which instruction comes next. Remember this: it will matter in the last part of today.

Five steps of every instruction

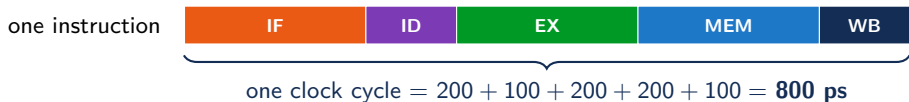


Each step uses a **different part** of the processor.

Single-cycle: one instruction per clock cycle

IF	ID	EX	MEM	WB
200 ps	100 ps	200 ps	200 ps	100 ps

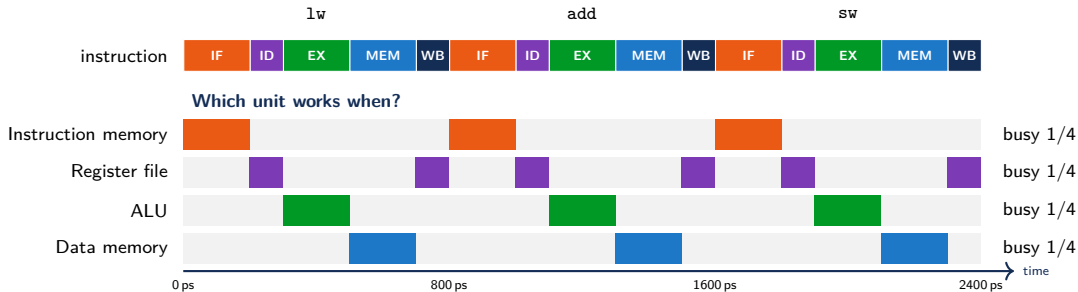
typical textbook values; 1 ps = 10^{-12} s



The whole instruction must fit into **one** clock cycle, so the cycle is long: **800 ps**.

One instruction at a time

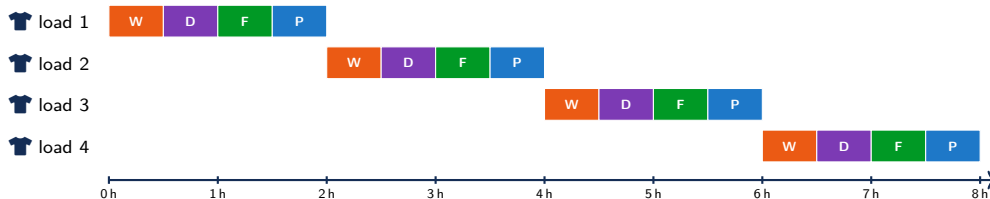
Three instructions on the **single-cycle** processor: $3 \times 800 \text{ ps} = 2400 \text{ ps}$.



Every unit is idle most of the time. Could these steps overlap?

The laundry problem

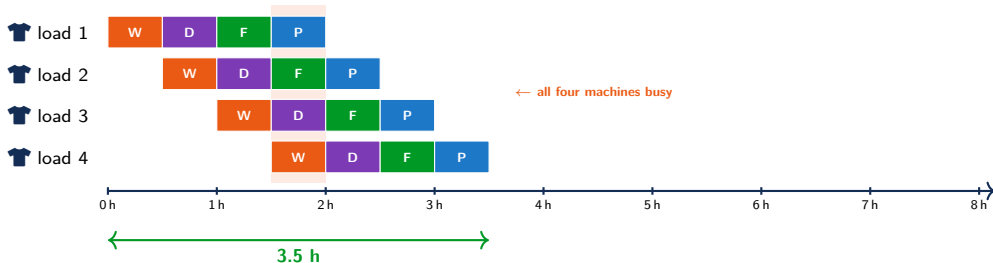
4 loads of laundry. Each load needs 4 steps of 30 minutes: **wash**, **dry**, **fold**, **put away**.



One load at a time: **8 hours**. The washing machine works only 1/4 of the time.

Decision: start the next load early

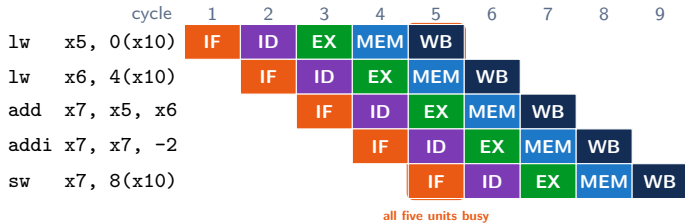
As soon as the washing machine is free, put the next load in.



3.5 hours. Each load still takes 2 hours, but one load is finished **every 30 minutes**.

Pipelining instructions

A new instruction starts every cycle. Each cycle, every instruction moves one stage forward.



IF fetch ID decode EX execute MEM memory WB write back

After the pipeline fills up, **one instruction finishes every cycle.**

Exercise 7: watch the pipeline



```
addi x5, x0, 1
addi x6, x0, 2
addi x7, x0, 3
addi x8, x0, 4
addi x9, x0, 5
```

x0 is always 0, so `addi x5, x0, 1` puts 1 into x5.

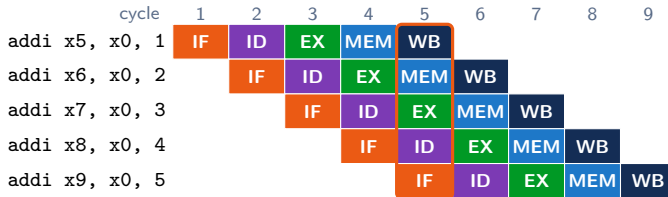
Ripes task

Choose the **5-stage processor** (). Run the program one step at a time and watch the **Editor**: the coloured bars show which instruction is in which stage.

1. What is the largest number of instructions in the processor at the same time?
2. After how many steps does the first instruction finish (WB)? And the last one?
3. How many cycles and instructions does Ripes count?

 [ksawerym.github.io/ComputerHardwareSimulators/#pipeline](https://github.com/ksawerym/ComputerHardwareSimulators/#pipeline)

Solution: watch the pipeline



1. At the same time
up to **5** instructions (step 5)

2. Finished
the first after **5** steps, the last after **9**

3. Ripes
9 cycles, **5** instructions

After the pipeline fills up, **one instruction finishes every cycle.**

Control hazard: which instruction comes next?



```
li    x11, 3
loop: addi x11, x11, -1
      bnez x11, loop
      addi x12, x0, 7
```

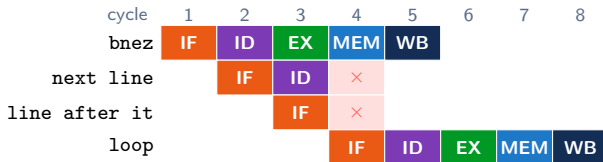
The problem

The pipeline fetches a new instruction **every cycle**. But after `bnez`: the next line, or back to `loop`?



When `bnez` finally knows the answer, **two more instructions** are already in the pipeline.

Fetch anyway, throw away if wrong



Simplest idea

Keep fetching the next lines, as if there were no jump.

If the jump happens

Throw the fetched instructions away (**flush**) and start again at loop.

Each jump costs **2 cycles**: two instructions were started for nothing.

Exercise 8: catch the flush



```
li    x11, 3           # counter = 3
loop: addi x11, x11, -1  # counter - 1
      bnez x11, loop    # repeat while != 0
      addi x12, x0, 7    # after the loop
```

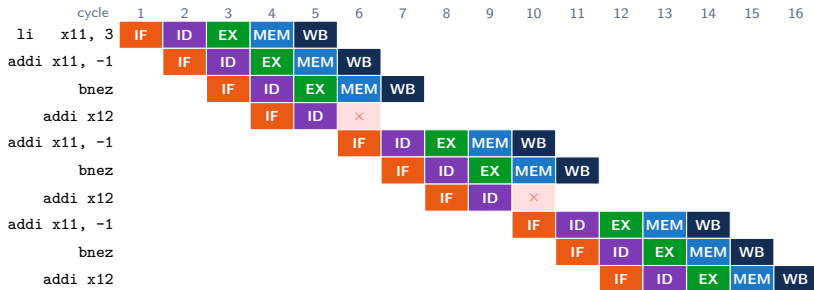
Ripes task

5-stage processor. Run the program one step at a time and watch the last line, `addi x12, x0, 7`, in the **Editor**.

1. How many times does `addi x12` enter the pipeline (IF) before it finally reaches WB?
2. What happens to it the other times? Why?
3. How many times does the loop jump back?

 [ksawerym.github.io/ComputerHardwareSimulators/#pipeline](https://github.com/Ksawerym/ComputerHardwareSimulators/#pipeline)

Solution: catch the flush



1–2. Three times

Twice it is **thrown away**: bnez decided to jump back to loop.

3. Two jumps

The third time $x11 = 0$, so there is no jump and addi x12 finishes.

Processors guess

Always “next line”

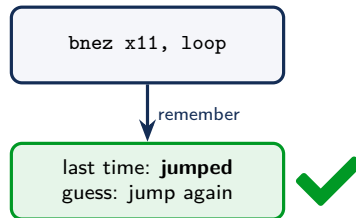
is a bad guess for a loop: a loop almost always jumps back.

Branch prediction

The processor remembers what each branch did **last time** and fetches from there.

In real processors

The guess is right **well over 90%** of the time.



A good guess means the loop loses cycles only once: when it finally **leaves** the loop.